Unified Engineering Workbench

Assessment prototype | C++17 | Windows 11 x64 target | 1 November 2026

Delivery status: a Windows x64 executable has been cross-compiled on Apple Silicon. Mac release builds, numerical tests and HTTP/browser workflows have been exercised. Actual Windows 11 launch is unverified. The candidate handwritten appendix is outstanding. These are explicit limitations, not claimed successes.

1. Design and reproducibility

A portable C++ executable serves a bundled browser interface over loopback only. C++ owns numerical work, parsers, benchmarks and hardware mock state. HTML/CSS/JavaScript provide presentation, an orthographic 3D view and plots. No external frontend or solver libraries are needed, and no CDN is used.

Layer	Implementation
Numerical core	Two-node rod FEA, tridiagonal solve, analytical comparison, explicit backpropagation
Input	Strict ASCII STL and five-column nodal CSV parsing in C++
Concurrency	Bounded numerical workers; independently bounded HTTP request pool; deterministic result ordering
Presentation	Geometry, Thermal, Performance, Hardware and Backpropagation views; validation, status, errors, exports
Delivery	CMake source build; Windows Launch.exe; VS Code task; relative runtime asset paths

After ZIP extraction, Launch.exe starts the backend and opens the default browser. The console remains open and closing it stops the app. Runtime data and web paths are resolved relative to the executable, rather than the current working directory. Windows system DLLs are not redistributed; MinGW/GCC runtime code is statically linked.

See README.md for build/test commands, data/PROVENANCE.md for inputs, dependencies.lock.json and LICENSES.md for dependencies, and the Git bundle for actual history. Verification & evidence reruns ten C++ checks with measured tolerances. docs/RUBRIC_REVIEW.md maps all twelve criteria to evidence and pending gates. Windows acceptance and candidate defence guides supplement the typed handwritten-appendix guide.

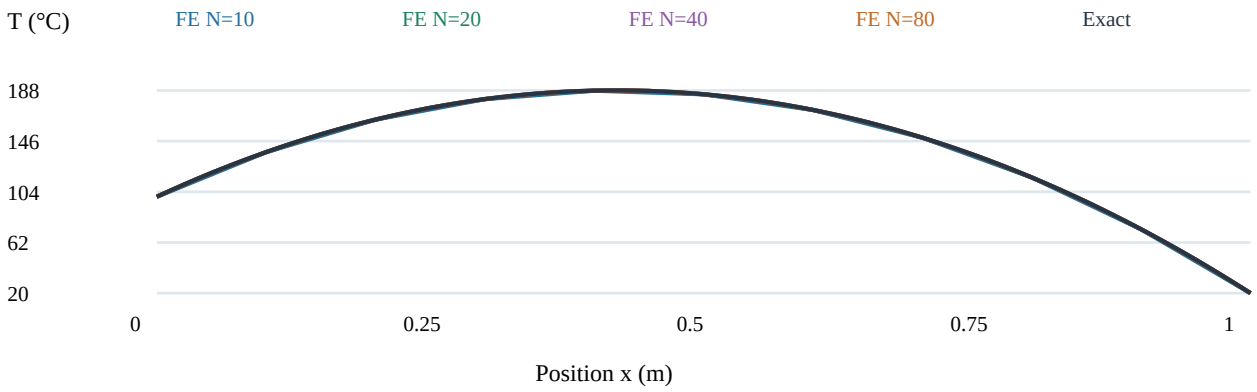
2. Thermal formulation and verification

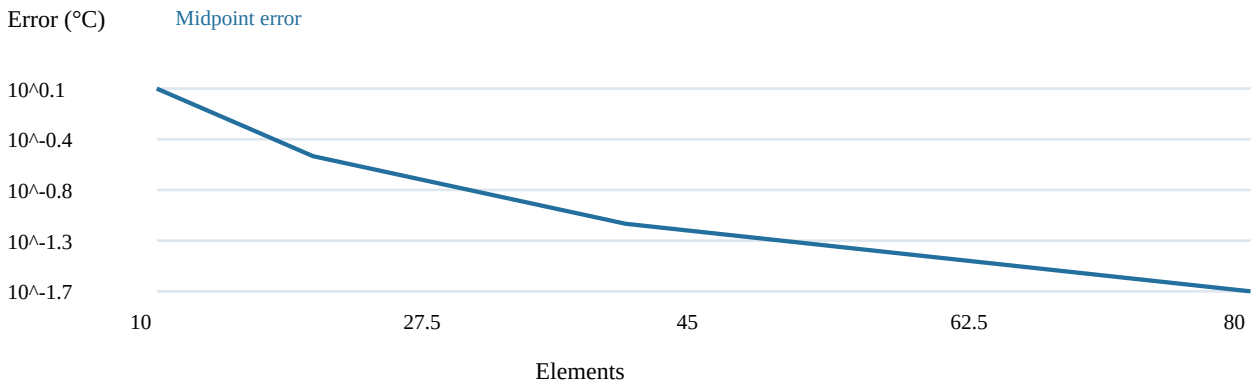
Assumptions: uniform rod, steady state, constant k and A, uniform volumetric source q, insulated lateral surface, prescribed temperatures at both ends. Defaults: L=1 m, A=1e-4 m², k=10 W/(m K), q=10000 W/m³; T(0)=100 °C and T(L)=20 °C.

For element length h: K_e=(kA/h)[[1,-1],[-1,1]], f_e=(qAh/2)[1,1]. Element contributions are assembled into a tridiagonal system. Prescribed boundary contributions are transferred into the neighbouring right-hand sides before boundary rows are replaced. Thomas elimination/back-substitution solves the system in O(N) time.

T_exact(x)=100-80x/L+[q/(2k)]x(L-x). The FE field is interpolated at each midpoint before computing the maximum absolute error. Checking nodes alone would hide interpolation error for this uniform, constant-source problem.

Elements	Max midpoint error (°C)	Previous/current
10	1.250000000	-
20	0.312500000	4.000000
40	0.078125000	4.000000
80	0.019531250	4.000000





Observed convergence is second order: halving h reduces error by four. Independent expected midpoint error is |q|h²/(8k). Boundary, zero-source linear, non-default-temperature and invalid-parameter tests supplement analytical agreement. Raw nodal and convergence CSVs are included.

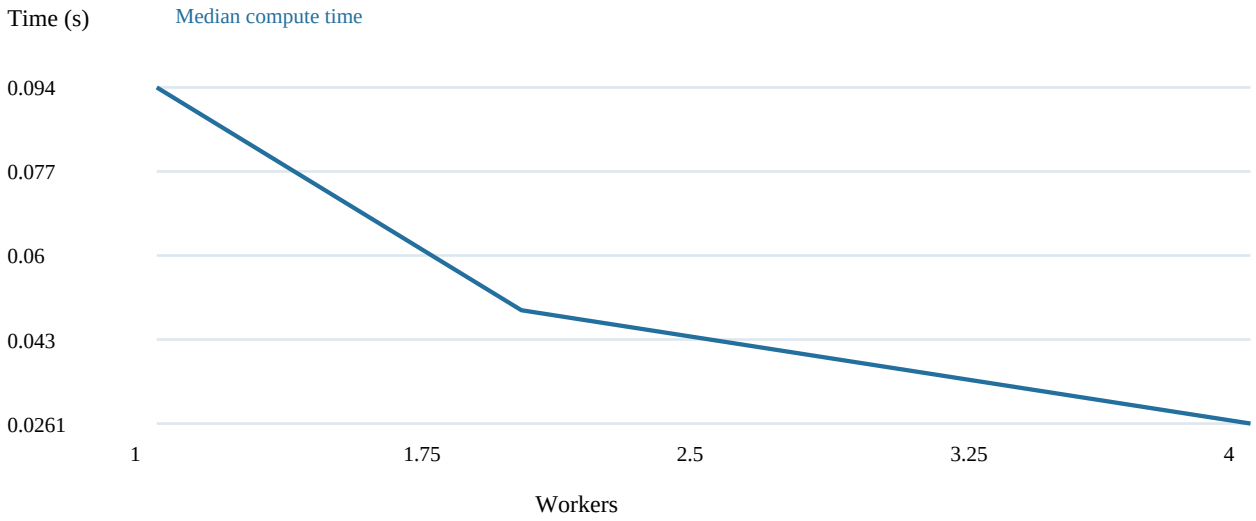
3. Reproducible multicore experiment

Measured on Apple M5, 10 physical / 10 logical CPUs reported by macOS, 16 GiB RAM. Compiler: Apple Clang 21.0.0; CMake Release flags -O3 -DNDEBUG, C++17. These measurements are from the Mac and are not Windows timing claims.

Workload: 2000 independent cases, 5000 elements each. mt19937 seed 20261101; explicit integer-to-double mapping produces k in [2,30] and q in [100,20000]. The serial and parallel paths call the identical solver on identical input cases. An atomic index distributes independent cases into fixed output slots.

Protocol: one warm-up per worker count and five timed repetitions; steady_clock wall time; median reported. Timings include thread startup, allocation, solver diagnostics and result storage. Generation, agreement comparison, file loading, rendering and compilation are excluded. Every node is compared outside timing. UI runs also incur optional atomic progress-counter overhead; CLI evidence omits those counters.

Workers	Median (s)	Speedup	Efficiency	Max diff (°C)
1	0.094004	1.000	1.000	0.0e+00
2	0.048974	1.919	0.960	0.0e+00
4	0.026065	3.606	0.902	0.0e+00



All observed serial/parallel differences are zero, below 1e-8 °C. Scaling is affected by allocation, memory bandwidth, worker scheduling and thread startup. Shorter workloads amplify overhead; no minimum speedup is assumed. A single-logical-CPU system can only report its supported path and cannot establish multicore speedup. Raw five-repetition timing CSV is supplied. Launch-to-ready measurements are recorded separately in results/generated/launch_timing.json.

4. Geometry, input validation and tests

ASCII STL is parsed as explicit solid/facet/normal/outer-loop/three-vertex/endloop/endfacet/endsolid records in C++. Binary STL and native CAD solids are outside scope. CSV requires the exact header node_id,x_m,y_m,z_m,temperature_C and five finite numeric fields, with unique non-negative integer IDs.

The self-created tetrahedron has four triangles; its intended units are metres because STL itself has no unit metadata. The supplied temperature CSV has ten synthetic nodes following the analytical rod profile. It is labelled synthetic and is not presented as an actual ANSYS export. No mesh-to-result registration is performed.

The custom orthographic renderer supports rotation, pan, zoom and fit. Triangles are depth-sorted; temperature points have a colour legend, coordinate units and clickable ID/location/temperature inspection. It is a small software renderer rather than a CAD engine.

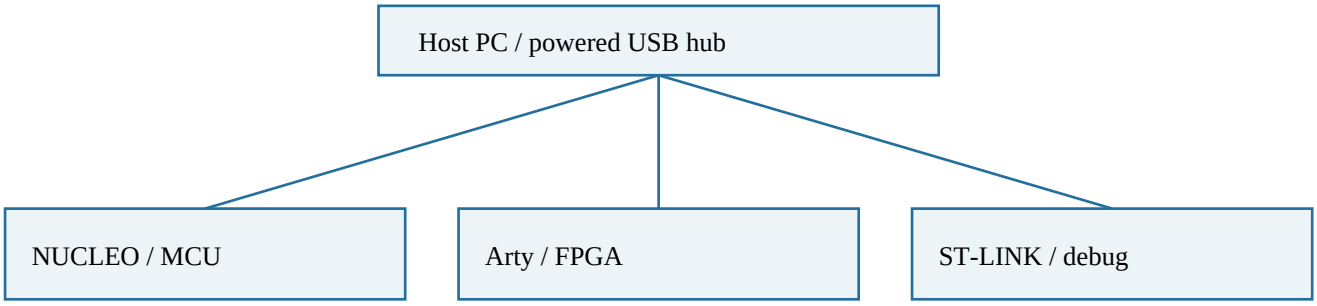
Validation area	Evidence
Numerical	Boundary conditions, analytical nodal agreement, midpoint error formula, convergence ratios, zero-source linear case
Negative imports	Empty input, no triangles/nodes, malformed structures, wrong fields, non-finite values, duplicate IDs and trailing numeric garbage
Input limits	20 MiB per file; 10000 triangles or nodes; coordinate magnitude limits; tested boundary and oversize cases
Concurrency	Five repetitions, serial/parallel agreement across all nodal values, deterministic case order
Integration	Health, custom/default solves, imports, invalid inputs, benchmark, three mock recovery sequences, gradients, traversal and cross-origin rejection

Versions 1.1-1.3 add progress/cancellation/restart, thermal diagnostics, custom studies, mesh comparison, a C++ probe and a searchable node explorer. A dedicated verification/evidence view reruns ten C++ checks and links the full rubric audit and acceptance guides. The current suite has 334 core checks and 23 API scenarios, including raw malformed HTTP tests. No runtime badge is a company score or Windows launch certificate.

Review fixes included preserving timeout messages, removing argument-spread extrema on large plots, bounding HTTP concurrency and replacing implementation-dependent random distributions. Windows runtime validation remains pending; an executable import-table check is not substituted for a real launch test.

5. Embedded design and recovery

A dedicated thread-safe C++ registry represents NUCLEO-F401RE, Arty A7-35T and ST-LINK/V2. All behaviour is simulated. Discovery exposes distinct capabilities. MCU temperature, FPGA counters and debug-probe status have sequenced mock telemetry. Sequences persist across reconnects. A bounded 100-event timestamped history is exportable. Tests cover all devices, concurrent telemetry, timeout recovery and history limits.



Link	Electrical/interface assumption
Nucleo CN1 USB	USB power and onboard VCP bridge; MCU domain 3.3 V; USART2 PA2/PA3 routing checked per revision
Arty J10 USB	Onboard USB UART/JTAG; board-specific interface; FPGA UART firmware/logic is not supplied
ST-LINK CN3 to STM32	VAPP pin 1 senses target 3.3 V; GND pin 4; SWDIO pin 7; SWCLK pin 9; NRST pin 15; pin 19 not used for target power
Signal ownership	Probe drives clock/control; target drives TDO in JTAG mode; SWDIO is bidirectional. One debug master per target.

ST-LINK is a debug bridge, not an interchangeable compute board. This design uses its SWD mode for the STM32. Arty retains its own supported debug bridge. Isolate the onboard STM32 debug master before attaching an external probe, verify target-side pin access, share ground, sense Vref and prevent power back-feed. Use level translation when domains differ. No physical wiring or programming was performed.

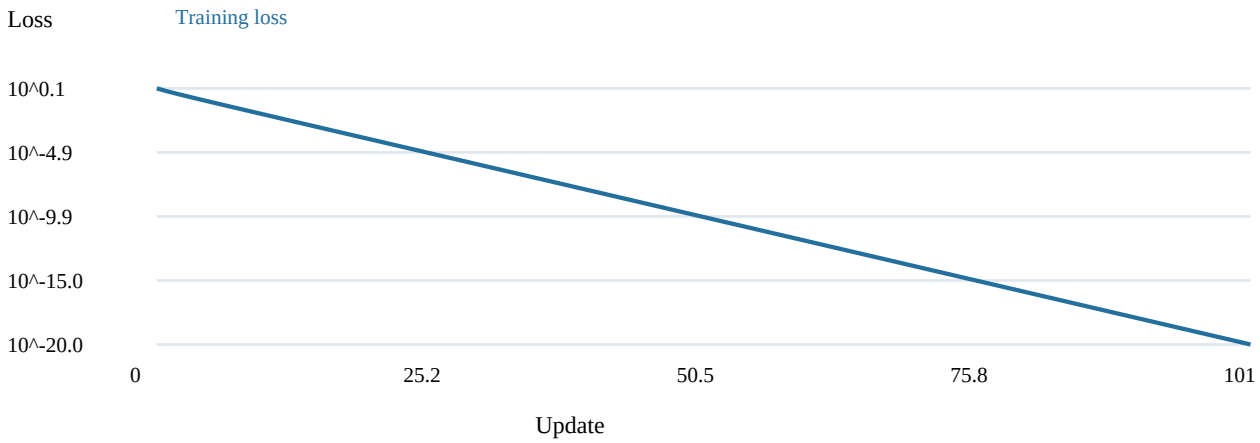
Vendor sources: ST UM1724 (Nucleo connectors), ST UM1075 Table 4 (ST-LINK pinout), Digilent Arty A7 E.2 schematic and Digilent-authored reference manual. Exact links, interface table, assumptions and safe integration sequence are in docs/EMBEDDED_DESIGN.md.

6. Backpropagation, learning and limitations

For x=1, y=2, w=0.5, b=0, v=1, c=0: z=wx+b; h=tanh(z); prediction=vh+c; J=0.5(prediction-y)². Initial hidden output is 0.462117157260 and loss is 1.182541818997. With e=prediction-y, gradients are e*v*(1-h²)*x, e*v*(1-h²), e*h and e for w,b,v,c respectively.

Parameter	Initial gradient	First simultaneous update
w	-1.209464475	0.620946448
b	-1.209464475	0.120946448
v	-0.710682047	1.071068205
c	-1.537882843	0.153788284

Central finite differences use epsilon=1e-6. Maximum gradient discrepancy is 1.130e-10, below 1e-5. All updates use gradients from the same parameter state. Initial negative gradients increase the parameters and raise an underprediction locally. The first updated loss is 0.685773765; 100 subsequent steps reduce it to 1.080e-20.



Learning insight: exact nodal temperatures do not prove an exact FE field; midpoint interpolation reveals error. Likewise numerical gradients independently check the chain rule, and parallel results require full-array comparison. Fitting one sample does not establish generalisation.

AI and ownership: substantial AI-generated code and documentation are disclosed. The candidate must review, understand and defend them. AI suggestions were corrected and tested; logs identify real examples. Actual personal effort and handwritten work are not fabricated. Outstanding items are actual Windows testing, candidate ID naming, handwritten appendix and candidate confirmation before upload.